<?php

MATERIAL DE ESTUDIO / DESARROLLO WEB

UNIDAD DIDÁCTICA 02

Tutorial 05
CRUD de usuarios

Base de datos MySQL, PDO y consultas preparadas

PHP PDO MySQL CRUD

APUNTES DE CLASE

Desarrollo web · PHP · Tutorial guiado

{ PHP }

UD 02 · DWES · PHP 8.1+

Tutorial 05 · CRUD de usuarios con PDO

Objetivo: Vamos a guardar usuarios en una base de datos MySQL. La aplicación creará cuentas, mostrará la lista, modificará nombres y eliminará registros. Separaremos la conexión, las consultas SQL y la página que recibe formularios.

Antes de comenzar

Utiliza PHP 8.1 o posterior y un servidor web local (por ejemplo, Apache de XAMPP). Crea una carpeta de proyecto dentro de `htdocs/UD2/` y **escribe los archivos con el contenido mostrado en cada paso**. El directorio `Codigo/` contiene esos mismos archivos listos para ejecutar.

Inicia **Apache y MySQL** desde XAMPP. Comprueba que PHP dispone de la extensión `pdo_mysql`. En los ejemplos se usa MySQL local con usuario `root` y contraseña vacía, una configuración habitual de laboratorio: si tu instalación utiliza otras credenciales, cámbialas en `config.php`. No utilices esas credenciales de prueba en un servidor público.

Crea `htdocs/UD2/Usuarios/`. Los archivos serán `schema.sql`, `config.php`, `UsuarioRepositorio.php` e `index.php`. Abre la aplicación con `http://localhost/UD2/Usuarios/index.php`.

Paso 1. Crear la tabla `usuario`

Abre phpMyAdmin (normalmente `http://localhost/phpmyadmin`), selecciona la pestaña SQL, pega el contenido de `schema.sql` y ejecútalo. Se crea la base de datos `ud2_usuarios` y la tabla `usuario`.

`id` es la clave primaria autonumérica, `email` es único, `nombre` es un texto obligatorio y `clave_hash` es el hash de la contraseña, no la contraseña en claro. `VARCHAR(255)` deja espacio para los hashes generados por PHP.

Archivo `Codigo/schema.sql`

```
CREATE DATABASE IF NOT EXISTS ud2_usuarios
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE ud2_usuarios;

CREATE TABLE IF NOT EXISTS usuario (
    id INT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(160) NOT NULL UNIQUE,
    nombre VARCHAR(120) NOT NULL,
    clave_hash VARCHAR(255) NOT NULL
);
```

Puedes comprobar la estructura desde phpMyAdmin, pero todavía no verás registros: se añadirán desde la aplicación web.

Paso 2. Centralizar la conexión en `config.php`

Crea `config.php`. La función `conectar(): PDO` devuelve una conexión con MySQL. La cadena DSN indica el servidor, la base de datos y la codificación `utf8mb4`. Los dos argumentos siguientes son usuario y contraseña.

Las opciones hacen que los errores de SQL lancen excepciones (`ERRMODE_EXCEPTION`), que las filas se recuperen como arrays asociativos (`FETCH_ASSOC`) y que los parámetros preparados se procesen realmente en MySQL (`EMULATE_PREPARES => false`).

Archivo `Codigo/config.php`

```
<?php
declare(strict_types=1);

function conectar(): PDO
{
    return new PDO(
        'mysql:host=127.0.0.1;dbname=ud2_usuarios;charset=utf8mb4',
        'root',
        '',
        [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false
        ]
    );
}
```

En este archivo no se consulta ninguna tabla ni se imprime nada. Su única responsabilidad es preparar y devolver la conexión PDO.

Paso 3. Escribir las operaciones en `UsuarioRepositorio.php`

Crea `UsuarioRepositorio.php`. El constructor recibe el objeto `PDO` y lo guarda en una propiedad privada. `listar()` ejecuta un `SELECT` y devuelve un array con `id`, `email` y `nombre`; deliberadamente no devuelve `clave_hash`.

`crear()` valida el correo, comprueba que el nombre no esté vacío y que la contraseña tenga al menos ocho caracteres; llama a `password_hash()` y usa una consulta preparada `INSERT`. `renombrar()` usa `UPDATE` para cambiar solo el nombre del ID indicado. `borrar()` ejecuta `DELETE` sobre ese ID.

Las operaciones de escritura utilizan `prepare()` y `execute()` con `?` para separar los valores del texto SQL. Los métodos devuelven datos o modifican la base de datos, pero **nunca hacen `echo`**.

Archivo `Codigo/UsuarioRepositorio.php`

```
<?php
declare(strict_types=1);

class UsuarioRepositorio
{
    public function __construct(private PDO $pdo) {}

    public function listar(): array
    {
        return $this->pdo
            ->query('SELECT id, email, nombre FROM usuario ORDER BY id DESC')
            ->fetchAll();
    }

    public function crear(string $email, string $nombre, string $password): void
    {
        $email = strtolower(trim($email));
        $nombre = trim($nombre);
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)
```

```
            || $nombre === '' || strlen($nombre) > 120 || strlen($password) < 8) {
            throw new InvalidArgumentException('Datos incorrectos; clave de al menos 8 caracteres.');
        }
        $hash = password_hash($password, PASSWORD_DEFAULT);
        $sql = 'INSERT INTO usuario(email, nombre, clave_hash) VALUES (?, ?, ?)';
        $sentencia = $this->pdo->prepare($sql);
        $sentencia->execute([$email, $nombre, $hash]);
    }

    public function renombrar(int $id, string $nombre): void
    {
        $nombre = trim($nombre);
        if ($id < 1 || $nombre === '' || strlen($nombre) > 120) {
            throw new InvalidArgumentException('Id o nombre no válido.');
        }
        $sentencia = $this->pdo->prepare('UPDATE usuario SET nombre = ? WHERE id = ?');
        $sentencia->execute([$nombre, $id]);
    }

    public function borrar(int $id): void
    {
        if ($id < 1) {
            throw new InvalidArgumentException('Id no válido.');
        }
        $sentencia = $this->pdo->prepare('DELETE FROM usuario WHERE id = ?');
        $sentencia->execute([$id]);
    }
}
```

El nombre de las columnas de PHP y SQL debe coincidir con `schema.sql`. El campo `clave_hash` solo se escribe al crear el usuario y nunca se muestra en el listado.

Paso 4. Crear formularios y mostrar el listado

Crea `index.php`. Al principio importa conexión y repositorio. La función `h()` utiliza `htmlspecialchars()` al representar nombres y correos. Dentro del `try` se crea el repositorio con una conexión válida.

Si la petición es `POST`, el campo oculto `accion` indica qué operación ejecutar: `crear`, `renombrar` o `borrar`. Para renombrar y borrar se valida que el ID sea un entero positivo. Después de una modificación correcta, `header('Location: index.php')` evita que el navegador reenvíe el formulario al actualizar la página.

En una petición `GET` se ejecuta `listar()` y se generan los elementos `<li>`. Cada usuario tiene su propio formulario para renombrarlo y otro para borrarlo, ambos con su ID oculto. Las excepciones por validación muestran un mensaje; los errores de PDO se presentan mediante un texto general que no revela credenciales.

Archivo `Codigo/index.php`

```
<?php
declare(strict_types=1);
require_once __DIR__ . '/config.php';
require_once __DIR__ . '/UsuarioRepositorio.php';

function h(string $valor): string
{
    return htmlspecialchars($valor, ENT_QUOTES, 'UTF-8');
}

$error = '';
$usuarios = [];
try {
    $repo = new UsuarioRepositorio(conectar());

    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
```

```
        $accion = (string) ($_POST['accion'] ?? '');
        $id = filter_var(
            $_POST['id'] ?? null,
            FILTER_VALIDATE_INT,
            ['options' => ['min_range' => 1]]
        );
        if ($accion === 'crear') {
            $repo->crear(
                (string) ($_POST['email'] ?? ''),
                (string) ($_POST['nombre'] ?? ''),
                (string) ($_POST['clave'] ?? '')
            );
        } elseif ($accion === 'renombrar' && $id !== false) {
            $repo->renombrar($id, (string) ($_POST['nombre'] ?? ''));
        } elseif ($accion === 'borrar' && $id !== false) {
            $repo->borrar($id);
        } else {
            throw new InvalidArgumentException('Acción no válida.');
        }
        header('Location: index.php');
        exit;
    }
    $usuarios = $repo->listar();
} catch (InvalidArgumentException $errorValidacion) {
    http_response_code(400);
    $error = $errorValidacion->getMessage();
} catch (PDOException $errorBD) {
    http_response_code(500);
    $error = 'No se pudo completar la operación. Revisa la conexión o los datos duplicados.';
}
?>
<!doctype html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>CRUD de usuarios</title>
</head>
<body>
    <h1>Usuarios</h1>
    <?php if ($error !== ''): ?>
        <p><?= h($error) ?></p>
    <?php endif; ?>

    <h2>Crear usuario</h2>
    <form action="index.php" method="post">
        <input type="hidden" name="accion" value="crear">
        <input type="email" name="email" placeholder="Correo" required>
        <input name="nombre" placeholder="Nombre" required>
        <input type="password" name="clave" minlength="8" placeholder="Contraseña" required>
        <button type="submit">Crear</button>
    </form>

    <h2>Listado</h2>
    <ul>
        <?php foreach ($usuarios as $usuario): ?>
            <li>
                <?= h($usuario['nombre']) ?> (<?= h($usuario['email']) ?>)
                <form action="index.php" method="post" style="display:inline">
                    <input type="hidden" name="accion" value="renombrar">
                    <input type="hidden" name="id" value="<?= (int) $usuario['id'] ?>">
                    <input name="nombre" value="<?= h($usuario['nombre']) ?>" required>
                    <button type="submit">Renombrar</button>
                </form>
                <form action="index.php" method="post" style="display:inline">
                    <input type="hidden" name="accion" value="borrar">
                    <input type="hidden" name="id" value="<?= (int) $usuario['id'] ?>">
                    <button type="submit">Borrar</button>
                </form>
```

```
            </li>
        <?php endforeach; ?>
    </ul>
</body>
</html>
```

Por ejemplo, al pulsar «Renombrar», llegan `accion=renombrar`, `id` y `nombre` en `$_POST`. `index.php` interpreta esos valores y llama a `$repo->renombrar(...)`, que ejecuta el SQL correspondiente.

Resultado y comprobación

1. Abre `index.php` y registra `ana@example.com`, nombre `Ana` y contraseña `claveAna123`. Debe aparecer en el listado.
2. En phpMyAdmin, consulta la tabla `usuario`: `clave_hash` contiene una cadena hash, no `claveAna123`.
3. Intenta registrar el mismo correo otra vez: MySQL lo impide mediante la restricción `UNIQUE` y se muestra un error general.
4. En la fila de Ana cambia el nombre a `Ana María` y pulsa «Renombrar». El nuevo nombre aparece en el listado.
5. Pulsa «Borrar» en su fila: el registro desaparece.

Lo importante: la base de datos mantiene los datos, `UsuarioRepositorio` concentra las consultas y `index.php` gestiona peticiones y HTML. Para publicar una aplicación administrativa real faltarían autenticación, autorización y protección CSRF.